

Transações e Controle de Concorrência

Considere as seguinte tabelas

```
CREATE TABLE Cliente (  
  CPF INTEGER PRIMARY KEY,  
  nome TEXT,  
);
```

```
CREATE TABLE Conta (  
  id INTEGER PRIMARY KEY,  
  CPF INTEGER,  
  saldo REAL,  
  FOREIGN KEY (CPF) REFERENCES Cliente(CPF)  
);
```

```
CREATE TABLE Transacao (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  origem INTEGER,  
  destino INTEGER,  
  valor REAL CHECK (valor > 0),  
  data TEXT DEFAULT CURRENT_TIMESTAMP,  
  FOREIGN KEY(origem) REFERENCES Conta(id),  
  FOREIGN KEY(destino) REFERENCES Conta(id)  
);
```

Considere as seguinte tabelas

```
CREATE TABLE Cliente (  
  CPF INTEGER PRIMARY KEY,  
  nome TEXT,  
);
```

```
CREATE TABLE Conta (  
  id INTEGER PRIMARY KEY,  
  CPF INTEGER,  
  saldo REAL,  
  FOREIGN KEY (CPF) REFERENCES Cliente(CPF)  
);
```

```
CREATE TABLE Transacao (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  origem INTEGER,  
  destino INTEGER,  
  valor REAL CHECK (valor > 0),  
  data TEXT DEFAULT CURRENT_TIMESTAMP,  
  FOREIGN KEY(origem) REFERENCES Conta(id),  
  FOREIGN KEY(destino) REFERENCES Conta(id)  
);
```

```
INSERT INTO Cliente (CPF, nome) VALUES (123, 'João');  
INSERT INTO Cliente (CPF, nome) VALUES (144, 'Maria');  
INSERT INTO Conta (id, CPF, saldo) VALUES (1, 123, 100.00); -- João começa com R$100  
INSERT INTO Conta (id, CPF, saldo) VALUES (2, 144, 50.00); -- Maria começa com R$50
```

Considere as seguinte tabelas

```
CREATE TABLE Cliente (  
  CPF INTEGER PRIMARY KEY,  
  nome TEXT,  
);
```

```
CREATE TABLE Conta (  
  id INTEGER PRIMARY KEY,  
  CPF INTEGER,  
  saldo REAL,  
  FOREIGN KEY (CPF) REFERENCES Cliente(CPF)  
);
```

```
CREATE TABLE Transacao (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  origem INTEGER,  
  destino INTEGER,  
  valor REAL CHECK (valor > 0),  
  data TEXT DEFAULT CURRENT_TIMESTAMP,  
  FOREIGN KEY(origem) REFERENCES Conta(id),  
  FOREIGN KEY(destino) REFERENCES Conta(id)  
);
```

```
INSERT INTO Cliente (CPF, nome) VALUES (123, 'João');  
INSERT INTO Cliente (CPF, nome) VALUES (144, 'Maria');  
INSERT INTO Conta (id, CPF, saldo) VALUES (1, 123, 100.00); -- João começa com R$100  
INSERT INTO Conta (id, CPF, saldo) VALUES (2, 144, 50.00); -- Maria começa com R$50
```

**Transfira 20 reais da conta do João
(CPF = 123) para a conta da Maria
(CPF = 144)**

Considere as seguinte tabelas

```
CREATE TABLE Cliente (  
  CPF INTEGER PRIMARY KEY,  
  nome TEXT,  
);
```

```
CREATE TABLE Conta (  
  id INTEGER PRIMARY KEY,  
  CPF INTEGER,  
  saldo REAL,  
  FOREIGN KEY (CPF) REFERENCES Cliente(CPF)  
);
```

```
CREATE TABLE Transacao (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  origem INTEGER,  
  destino INTEGER,  
  valor REAL CHECK (valor > 0),  
  data TEXT DEFAULT CURRENT_TIMESTAMP,  
  FOREIGN KEY(origem) REFERENCES Conta(id),  
  FOREIGN KEY(destino) REFERENCES Conta(id)  
);
```

```
INSERT INTO Cliente (CPF, nome) VALUES (123, 'João');  
INSERT INTO Cliente (CPF, nome) VALUES (144, 'Maria');  
INSERT INTO Conta (id, CPF, saldo) VALUES (1, 123, 100.00); -- João começa com R$100  
INSERT INTO Conta (id, CPF, saldo) VALUES (2, 144, 50.00); -- Maria começa com R$50
```

**Transfira 20 reais da conta do João
(CPF = 123) para a conta da Maria
(CPF = 144)**

Resposta:

```
UPDATE Conta  
SET saldo = saldo - 20  
WHERE CPF = 123
```

```
UPDATE Conta  
SET saldo = saldo + 20  
WHERE CPF = 144
```

```
INSERT INTO Transacao (origem, destino, valor)  
VALUES (  
  (SELECT id FROM Conta WHERE CPF = 123 LIMIT 1),  
  (SELECT id FROM Conta WHERE CPF = 144 LIMIT 1),  
  20.00  
);
```

--- retirando R\$ 20 da conta do João

```
UPDATE Conta  
SET saldo = saldo - 20  
WHERE CPF = 123
```

Vamos dar uma olhada bem detalhada no que está acontecendo no comando acima

--- retirando R\$ 20 da conta do João

```
UPDATE Conta  
SET saldo = saldo - 20  
WHERE CPF = 123
```

Vamos dar uma olhada bem detalhada no que está acontecendo no comando acima

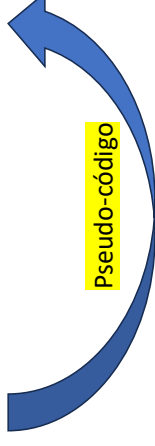
Para executar o comando acima, o SGBD realiza a seguinte sequência de operações:

- 1) Abre a tabela Conta e busca qual é o saldo do usuário cujo CPF = 123 e salva em um variável temporária X, ou seja, **read(X)**
- 2) Depois ele faz o seguinte cálculo: **X = X – 20**
- 3) Depois ele salva o novo valor da variável X no saldo do usuário, ou seja, **write(X)**

--- retirando R\$ 20 da conta do João

```
UPDATE Conta  
SET saldo = saldo - 20  
WHERE CPF = 123
```

```
read(x)  
x = x - 20  
write(x)
```



--- retirando R\$ 20 da conta do João

```
UPDATE Conta
SET saldo = saldo - 20
WHERE CPF = 123
```

```
read(x)
x = x - 20
write(x)
```

Pseudo-código

--- adicionando R\$ 20 na conta da Maria

```
UPDATE Conta
SET saldo = saldo + 20
WHERE CPF = 144
```

```
read(y)
y = y + 20
write(y)
```

Pseudo-código

- Problemas que poderemos ter:

- Durante a transferência, pode cair a energia do servidor onde está o banco de dados e não ser efetuado o crédito na conta da Maria.
Ou então, a operação não é registrada na tabela Transacao

- **Problemas que poderemos ter:**
 - Durante a transferência, pode cair a energia do servidor onde está o banco de dados e não ser efetuado o crédito na conta da Maria. Ou então, a operação não é registrada na tabela Transacao
- **Solução:** colocar os comandos em um bloco de Transação

→ BEGIN TRANSACTION;

*COLOCAR OS
COMANDOS AQUI*

→ COMMIT;

- **Problemas que poderemos ter:**
 - Durante a transferência, pode cair a energia do servidor onde está o banco de dados e não ser efetuado o crédito na conta da Maria. Ou então, a operação não é registrada na tabela Transacao
- **Solução:** colocar os comandos em um bloco de Transação



```
BEGIN TRANSACTION;  
  
UPDATE Conta SET saldo = saldo - 20 WHERE CPF = 123;  
  
UPDATE Conta SET saldo = saldo + 20 WHERE CPF = 144;  
  
INSERT INTO Transacao (origem, destino, valor) VALUES (  
    (SELECT id FROM Conta WHERE CPF = 123 LIMIT 1),  
    (SELECT id FROM Conta WHERE CPF = 144 LIMIT 1),  
    20.00 );  
  
COMMIT;
```

O que é transação?

- Uma transação é uma seqüência de operações SQL que forma uma unidade lógica de trabalho
- Toda transação deve satisfazer as propriedades **ACID**

<u>Propriedade</u>	<u>Descrição</u>	<u>Exemplo no caso</u>
A tomicidade	Todas as operações devem ser executadas integralmente ou nenhuma.	Se o <i>débito</i> de João ocorrer, o <i>crédito</i> em Maria também deve ocorrer.
C onsistência	O Banco de Dados deve permanecer em estado consistente antes e depois da transação.	A soma total dos saldos (R\$150) não muda.
I solamento	Transações simultâneas não interferem entre si.	Se duas transferências ocorrem ao mesmo tempo, uma não deve “ver” o estado parcial da outra.
D urabilidade	Após COMMIT, as alterações persistem mesmo após falha.	Mesmo que o sistema caia, os R\$20 já transferidos permanecem.

Vamos falar sobre a propriedade
Isolamento

Considere 2 transações

T1 = read(x), $x = x - 20$, write(x)

T2 = read(x), $x = x + 20$, write(x)

Vamos assumir que as duas transações estão mexendo **no mesmo x** (mesma linha/tabela)

Considere 2 transações

T1 = read(x), x = x - 20, write(x)
T2 = read(x), x = x + 20, write(x)

}
Vamos assumir que as duas transações estão mexendo **no mesmo x** (mesma linha/tabela)

Maneiras que podemos executar essas transações

Plano Sequencial (ou Serial)

Transação 1	Transação 2
Read(x) X = x - 20 Write(x) COMMIT	
	Read(x) X = x + 20 Write(x) COMMIT

Suponha que cada transação gaste 1 segundo.
Suponha também que as transações iniciem juntas.

A Transação 1 gastou 3 segundos
A Transação 2 gastou 6 segundos, pois teve que **esperar 3 segundos** até a Transação 1 finalizar

Porém, ninguém gosta de ficar esperando!

Considere 2 transações

T1 = read(x), x = x – 20, write(x)

T2 = read(x), x = x + 20, write(x)

Maneiras que podemos executar essas transações

Plano Concorrente (ou Intercalado)

Transação 1	Transação 2
Read(x)	
	Read(x) X = x + 20
X = x – 20 Write(x)	
	Write(x) COMMIT
COMMIT	

Plano Sequencial (ou Serial)

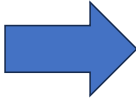
Transação 1	Transação 2
Read(x) X = x - 20 Write(x) COMMIT	
	Read(x) X = x + 20 Write(x) COMMIT

Considere 2 transações

T1 = read(x), x = x - 20, write(x)

T2 = read(x), x = x + 20, write(x)

Qual o problema
dessa execução?



Maneiras que podemos executar essas transações

Plano Sequencial (ou Serial)

Transação 1	Transação 2
Read(x) X = x - 20 Write(x) COMMIT	
	Read(x) X = x + 20 Write(x) COMMIT

Plano Concorrente (ou Intercalado)

Transação 1	Transação 2
Read(x)	
	Read(x) X = x + 20
X = x - 20 Write(x)	
	Write(x) COMMIT
COMMIT	

O fato é...
execuções concorrentes precisam de **bloqueios** para não
causar inconsistências

Mas...

Quando escrevemos apenas **BEGIN TRANSACTION** para
criar um bloco de transações **não** estamos especificando
qual é o tipo de bloqueio, estamos deixando para o
SGBD decidir **quando** e **quais** bloqueios utilizar conforme
o nível de isolamento definido por padrão.

Em **SQLite** todos esses comando são válidos para criar blocos de transações

- `BEGIN;`
- `BEGIN TRANSACTION;`
- `BEGIN DEFERRED;`
- `BEGIN IMMEDIATE;`
- `BEGIN EXCLUSIVE;`

Em SQLite todos esses comando são válidos para criar blocos de transações

- BEGIN;
- BEGIN TRANSACTION;
- BEGIN DEFERRED;
- BEGIN IMMEDIATE;
- BEGIN EXCLUSIVE;

Esses três comandos são similares.
As transações são iniciadas assim que o usuário executa-las, mas nenhum bloqueio é obtido imediatamente.
O bloqueio será solicitado quando ocorrer escrita.
É por isso que chamamos esse modo de *deferred* (“adiado”).

Exemplo

Transação 1	Transação 2
read(x)	
	read(x) X = x + 20
x = x - 20 lock_write write(x)	
	lock_write write(x) COMMIT
COMMIT	

Não vai conseguir, pois o banco de dados está bloqueado

Em SQLite todos esses comando são válidos para criar blocos de transações

- BEGIN;
- BEGIN TRANSACTION;
- BEGIN DEFERRED;
- BEGIN IMMEDIATE;
- BEGIN EXCLUSIVE;

Os dois são similares!

Serve para **reservar o bloqueio logo no primeiro comando de escrita (ou leitura)**, evitando que outra transação leia um valor antigo e depois sobrescreva o resultado da sua.

Exemplo

Transação 1	Transação 2
lock_write read(x)	
	lock_write read(x)
x = x - 20 write(x) COMMIT	
	x = x + 20 write(x) COMMIT

Não vai conseguir, pois o banco de dados está bloqueado por outra transação

Em resumo

•BEGIN;	São usados em
•BEGIN TRANSACTION;	transações que só
•BEGIN DEFERRED ;	tem leitura
•BEGIN IMMEDIATE ;	São usados em
•BEGIN EXCLUSIVE ;	transações que tem
	operações de escrita

Considere as seguinte tabelas

```
CREATE TABLE Aluno (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  nome TEXT NOT NULL  
);
```

```
CREATE TABLE Livro (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  titulo TEXT NOT NULL,  
  disponivel INTEGER NOT NULL DEFAULT 1 -- 1 = sim, 0 = não  
);
```

```
CREATE TABLE Emprestimo (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  id_aluno INTEGER NOT NULL,  
  id_livro INTEGER NOT NULL,  
  data_emprestimo TEXT DEFAULT CURRENT_TIMESTAMP,  
  FOREIGN KEY (id_aluno) REFERENCES Aluno(id),  
  FOREIGN KEY (id_livro) REFERENCES Livro(id)  
);
```

```
INSERT INTO Aluno (nome) VALUES ('João'), ('Maria');  
INSERT INTO Livro (titulo) VALUES  
  ('Banco de Dados'),  
  ('Engenharia de Software'),  
  ('Python para Todos');
```


Considere as seguinte tabelas

```
CREATE TABLE Aluno (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  nome TEXT NOT NULL  
);
```

```
CREATE TABLE Livro (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  titulo TEXT NOT NULL,  
  disponivel INTEGER NOT NULL DEFAULT 1 -- 1 = sim, 0 = não  
);
```

```
CREATE TABLE Emprestimo (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  id_aluno INTEGER NOT NULL,  
  id_livro INTEGER NOT NULL,  
  data_emprestimo TEXT DEFAULT CURRENT_TIMESTAMP,  
  FOREIGN KEY (id_aluno) REFERENCES Aluno(id),  
  FOREIGN KEY (id_livro) REFERENCES Livro(id)  
);
```

```
INSERT INTO Aluno (nome) VALUES ('João'), ('Maria');  
INSERT INTO Livro (titulo) VALUES  
  ('Banco de Dados'),  
  ('Engenharia de Software'),  
  ('Python para Todos');
```

**João pegou o livro Banco de Dados
emprestado. Qual é a transação?**

Considere as seguinte tabelas

```
CREATE TABLE Aluno (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  nome TEXT NOT NULL  
);
```

```
CREATE TABLE Livro (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  titulo TEXT NOT NULL,  
  disponivel INTEGER NOT NULL DEFAULT 1 -- 1 = sim, 0 = não  
);
```

```
CREATE TABLE Emprestimo (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  id_aluno INTEGER NOT NULL,  
  id_livro INTEGER NOT NULL,  
  data_emprestimo TEXT DEFAULT CURRENT_TIMESTAMP,  
  FOREIGN KEY (id_aluno) REFERENCES Aluno(id),  
  FOREIGN KEY (id_livro) REFERENCES Livro(id)  
);
```

```
INSERT INTO Aluno (nome) VALUES ('João'), ('Maria');  
INSERT INTO Livro (titulo) VALUES  
  ('Banco de Dados'),  
  ('Engenharia de Software'),  
  ('Python para Todos');
```

João pegou o livro Banco de Dados emprestado. Qual é a transação?

```
BEGIN TRANSACTION;
```

```
INSERT INTO Emprestimo (id_aluno, id_livro)  
VALUES (  
  (SELECT id FROM Aluno WHERE nome = 'João'),  
  (SELECT id FROM Livro WHERE titulo = 'Banco de Dados'),  
);
```

```
UPDATE Livro SET disponivel = 0  
WHERE id = (SELECT id FROM Livro WHERE titulo = 'Banco de  
Dados');
```

```
COMMIT;
```

Exercício nº 1

- Considere as seguintes tabelas

Aluno(mat, nome, #id)

Curso(id, nome)

Disciplina(cod, sigla, #id)

Matricula(#mat, #cod, semestre)

Matricule o aluno João na disciplina SBD1 do
curso de ESW no semestre 2026/1

Exercício nº 1

- Considere as seguintes tabelas

Aluno(mat, nome, #id)

Curso(id, nome)

Disciplina(cod, sigla, #id)

Matricula(#mat, #cod, semestre)

Matricule o aluno João na disciplina SBD1 do curso de ESW no semestre 2026/1

Resposta:

```
BEGIN TRANSACTION;

INSERT INTO Matricula (mat, cod, semestre)
VALUES (
    (SELECT id FROM Aluno WHERE nome = 'João'),
    (SELECT d.id
     FROM Disciplina d
     JOIN Curso c ON c.id = d.id_curso
     WHERE d.nome = 'SBD1' AND c.nome = 'ESW'),
    '2026/1'
);

COMMIT;
```

Exercício nº 2

- Considere as seguintes tabelas

Aluno(mat, nome, #id)

Curso(id, nome)

Disciplina(cod, sigla, #id)

Matricula(#mat, #cod, semestre)

Matricule todos os alunos do curso de ESW na disciplina de Felicidade no semestre 2026/1

Exercício nº 2

- Considere as seguintes tabelas

Aluno(mat, nome, #id)

Curso(id, nome)

Disciplina(cod, sigla, #id)

Matricula(#mat, #cod, semestre)

Matricule todos os alunos do curso de ESW na disciplina de Felicidade no semestre 2026/1

Resposta:

```
BEGIN TRANSACTION;
```

```
INSERT INTO Matricula (mat, cod, semestre)
```

```
SELECT a.mat, d.cod, '2026/1'
```

```
FROM Aluno a
```

```
JOIN Curso c ON a.id = c.id
```

```
JOIN Disciplina d ON d.id = c.id
```

```
WHERE c.nome = ESW' AND d.sigla = 'Felicidade';
```

```
COMMIT;
```

Exercício nº 3

- Considere as seguintes tabelas

Aluno(mat, nome, #id)

Curso(id, nome)

Disciplina(cod, sigla, #id)

Matricula(#mat, #cod, semestre)

Remova a disciplina Felicidade do curso de ESW e remova todas as matriculas associadas a esta disciplina para o semestre 2026/1

Exercício nº 3

- Considere as seguintes tabelas

Aluno(mat, nome, #id)

Curso(id, nome)

Disciplina(cod, sigla, #id)

Matricula(#mat, #cod, semestre)

Remova a disciplina Felicidade do curso de ESW e remova todas as matriculas associadas a esta disciplina para o semestre 2026/1

Resposta:

```
BEGIN TRANSACTION;
```

```
DELETE FROM Matricula
```

```
WHERE cod = (
```

```
  SELECT d.cod
```

```
  FROM Disciplina d
```

```
  JOIN Curso c ON c.id = d.id
```

```
  WHERE d.sigla = 'Felicidade' AND c.nome = 'ESW'
```

```
)
```

```
AND semestre = '2026/1';
```

```
DELETE FROM Disciplina
```

```
WHERE sigla = 'Felicidade'
```

```
AND id = (SELECT id FROM Curso WHERE nome = ESW);
```

```
COMMIT;
```


Exercício nº 4

- Considere as seguintes tabelas

Aluno(mat, nome, #id)

Curso(id, nome)

Disciplina(cod, sigla, #id)

Matricula(#mat, #cod, semestre)

Matricule todos os alunos de SBD1 do curso de ESW do semestre 2025/2 na disciplina de SBD2 de ESW no semestre de 2026/1

Exercício nº 4

- Considere as seguintes tabelas

Aluno(mat, nome, #id)

Curso(id, nome)

Disciplina(cod, sigla, #id)

Matricula(#mat, #cod, semestre)

Matricule todos os alunos de SBD1 do curso de ESW do semestre 2025/2 na disciplina de SBD2 de ESW no semestre de 2026/1

Resposta:

```
BEGIN TRANSACTION;  
  
INSERT INTO Matricula (mat, cod, semestre)  
SELECT  m.mat, d2.cod, '2026/1'  
FROM Matricula m  
JOIN Disciplina d1 ON d1.cod = m.cod  
JOIN Aluno a   ON a.mat = m.mat  
JOIN Curso c   ON c.id = a.id  
JOIN Disciplina d2 ON d2.id = c.id AND d2.sigla = 'SBD2'  
WHERE c.nome = 'ESW'  
AND d1.sigla = 'SBD1'  
AND m.semestre = '2025/2';  
  
COMMIT;
```